

Smart Guitar Amp

Analog signal chain + Arduino DSP firmware + voice / LLM tone control

Presented by · Jaehyung Choi / Gijeong Lee

Agenda

1

Live demo

Tuner · manual knobs · voice → LLM control

2

Circuit / hardware

Analog front-end, optical VGA, ADC, PWM reconstruction

3

Firmware / DSP

Real-time effects, AMDF tuner, auto-gain on an ATmega328P

4

Backend

FastAPI · Whisper STT · LLM tone engine · serial link

SWITCH TO THE APP

LIVE DEMO

1

Guitar tuner

Pluck a string — AMDF pitch detection, needle snaps to EADGBE

2

Manual control

Footswitch effects + draggable knobs drive the amp in real time

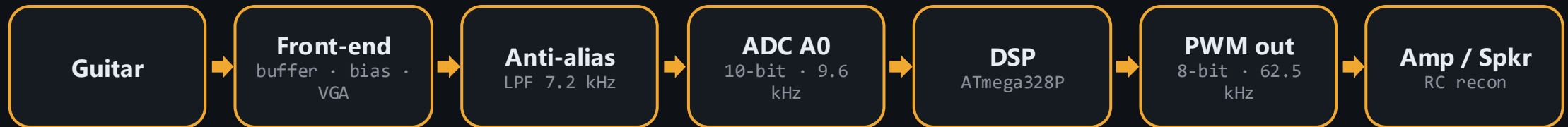
3

Voice → LLM

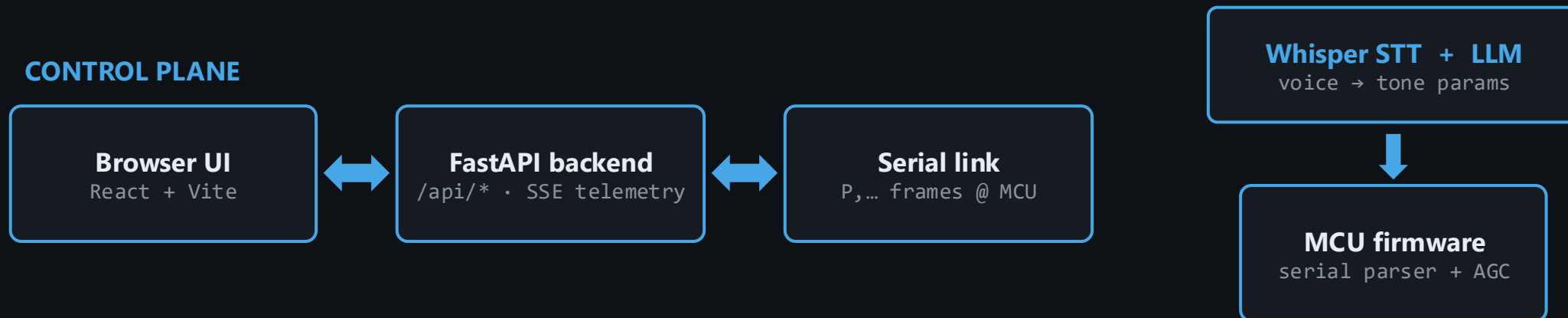
“hey amp” → Whisper → tone engine sets the effect for you

System Overview — three layers, one signal path

SIGNAL PATH



CONTROL PLANE



The amp runs DSP on the bare-metal MCU; the browser, FastAPI host, and LLM steer it over serial — without ever touching the real-time audio loop.

SECTION 01

Circuit / Hardware

Analog signal conditioning — guitar level in, line level out

Analog signal chain

INPUT → signal flows left to right → OUTPUT



Single 5V rail · bipolar audio rides a 2.5V virtual ground · all gain set optically (no digital noise into the analog path)

Input stage — bias & AC coupling

COMPONENTS

R1 — 10 k Ω

Series input resistor → high-Z input ($\sim M\Omega$ with op-amp), won't load the pickup

C1 — 10 μF

AC-coupling cap: blocks the guitar's DC, passes only the audio swing

2.5 V virtual ground

Mid-rail bias so a single 5 V supply carries a bipolar ($\pm$) audio signal

WHY 2.5 V BIAS?

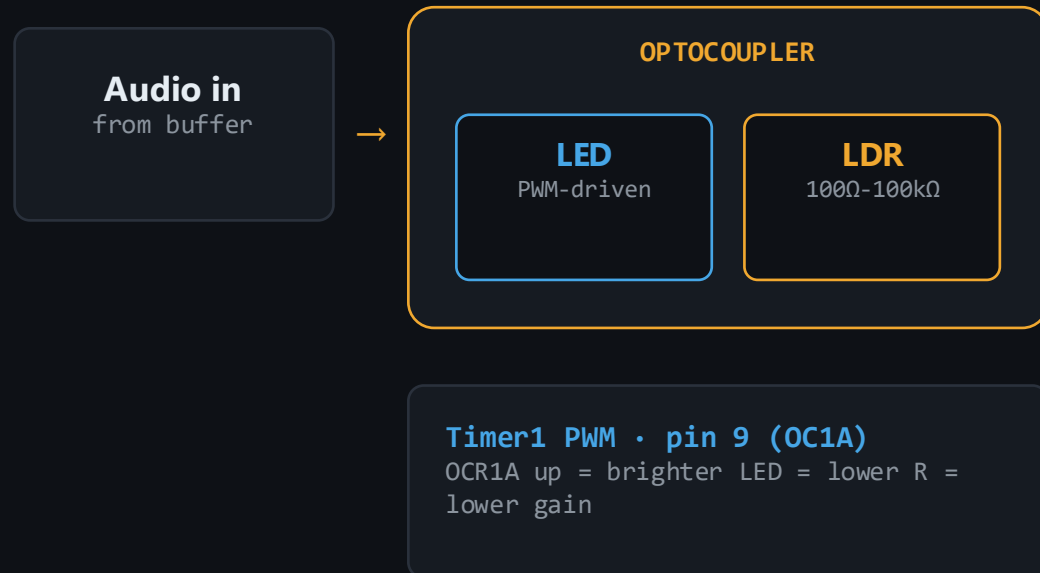
The Uno runs on one 5 V rail — it can't represent a negative voltage. Centring the signal at 2.5 V lets the waveform swing up and down around mid-rail, so the ADC sees the full $\pm$ audio without a split supply.

OP-AMP CHOICE

LM358 / MCP6002 — rail-to-rail, swings near 0–5 V on a single supply

Optocoupler VGA — silent gain

SIGNAL FLOW



WHY OPTICAL?

DC-free gain control

Light couples LED to LDR — no electrical path into the audio, so no DC offset injected.

Silent & click-free

Resistance glides smoothly — no zipper noise like a digital pot or switched gain.

No switching noise

PWM stays on the LED side; the analog path never sees the digital edges.

Anti-alias filter & ADC

FILTER & SAMPLING

Sallen-Key LPF

$R7 = R8 = 10\text{ k}\Omega$ · $C5 = C6 = 2.2\text{ nF}$
 $f_c \approx 7.2\text{ kHz}$ (2nd-order)

ADC input

A0 · 10-bit · free-running
prescaler 128 → 9615 Hz

Nyquist

$F_s / 2 = 4.8\text{ kHz}$
covers full guitar range

Guitar band sits safely below both the filter corner and Nyquist.

WHY PRESCALER 128, NOT 64?

Slower clock = longer echoes

64 would sample at ~19 kHz. Halving to 128 (9.6 kHz) doubles the time the 512-byte buffer spans — max delay/reverb roughly doubles.

Nyquist still covers guitar

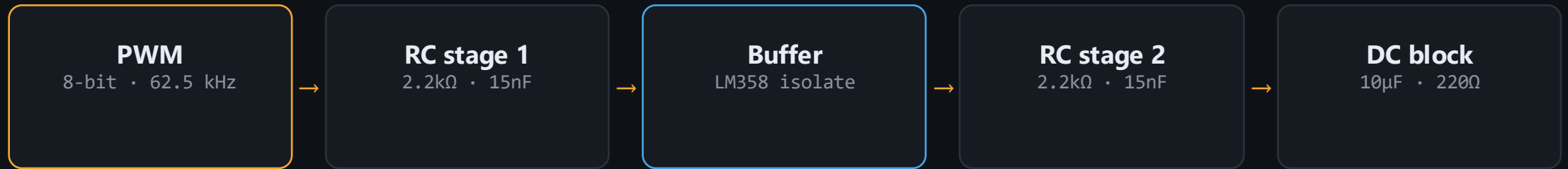
4.8 kHz Nyquist is well above the guitar's top harmonics, so nothing audible is lost by going slower.

Filter guards the band

The 7.2 kHz Sallen-Key rolls off anything that could alias back into the audio.

PWM output & reconstruction

RECONSTRUCTION CHAIN • pin 10 / OC1B



each RC ≈ 4.8 kHz/stage $\rightarrow$ ~ -44 dB at carrier, flat guitar passband • HPF ≈ 0.16 Hz

Timer1 – Mode 5, 8-bit Fast PWM

```
TCCR1A = (1<<COM1A1)|(1<<COM1B1)|(1<<WGM10);
TCCR1B = (1<<WGM12)|(1<<CS10); // /1  $\rightarrow$  62.5 kHz
OCR1B = 128; // mid-rail idle

// in the ADC ISR:
OCR1B = processEffect(ADC) >> 2;
```

WHY 62.5 kHz?

Not 15.6 kHz

Carrier far above audio means a gentle passive RC can reject it AND pass the full guitar band.

Trade: 10-bit $\rightarrow$ 8-bit

256 levels (~ 48 dB) is plenty for a guitar effect.

SECTION 02

Firmware / DSP

Real-time fixed-point effects on an ATmega328P

Real-time architecture — the ISR

ADC ISR • fires ~9 615x/sec • ~52 µs budget per sample



`struct Params` — written by `loop()`, read by `ISR`

```
uint8_t effect;    // FX_CLEAN ... FX_TUNER
int16_t thresh;    // overdrive drive 1..511
uint16_t delayLen; // tap distance 1..512
uint8_t feedback;  // regen Q8 0..255
uint8_t mix;       // wet level Q8 0..255
uint8_t depth;     // chorus depth 1..48
uint8_t rate;      // chorus LFO rate 1..20
```

DESIGN RULES

Fixed-point only

No float in the ISR — every effect is integer shifts & adds.

512-byte ring buffer

audioBuf holds ~53 ms of history for delay / reverb / chorus.

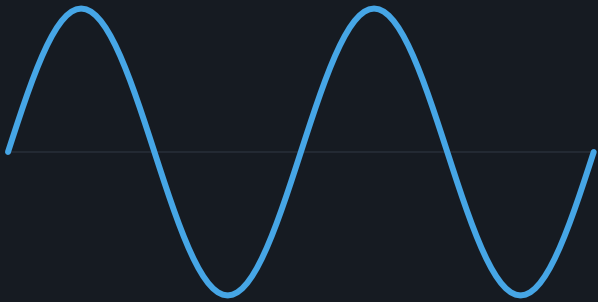
Atomic updates

Params swapped in one `cli/sei` block — ISR never reads a half-write.

Overdrive — pre-amplify, then hard-clip

CLEAN

thresh ≤ 16 · just at breakup



DRIVE = 180 (rock)

11× gain · odd harmonics appear



DRIVE = 480 (metal)

30× gain · near square wave



`driven = (x * thresh) >> 4;` hard-clip to `±511` → sets `g_clip` when railed

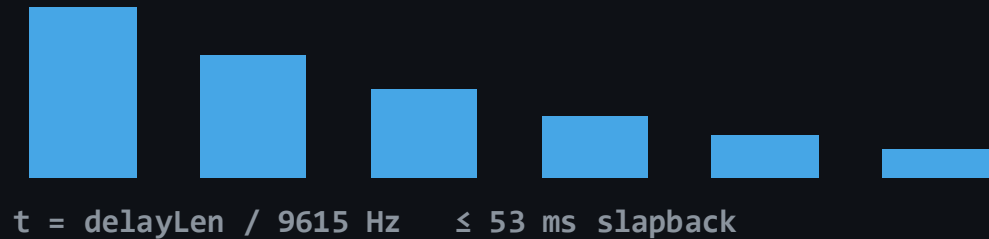
thresh/16 is the gain factor — 16× = edge of breakup, 180× = rock, 480× = extreme saturation. More clipping squares the wave → more odd harmonics.

Delay — ring-buffer echo

512-SAMPLE RING BUFFER



DECAYING TAIL (feedback regenerates)



Per-sample, in the ISR

```
echo = buf[(w - delayLen) & MASK]
out  = x + (echo.mix >> 8) // wet
store = x + (echo.feedback >> 8) // tail
buf[w] = store >> 2; w = (w+1) & MASK
```

HOW IT WORKS

One read tap, delayLen samples behind the write head, gives the echo.

Feeding the echo back into the store builds a recursive, decaying repeat — slapback to long trails.

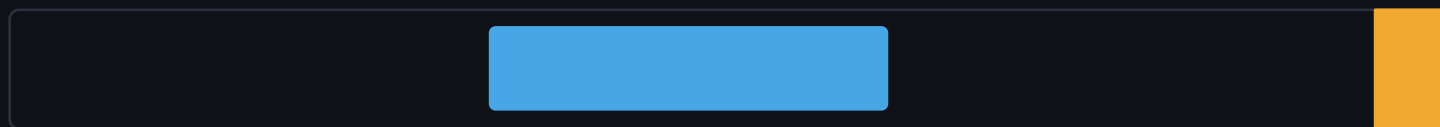
Chorus — modulated tap

TRIANGLE LFO



period 65536 · phase += rate (1-20) → slow, smooth sweep

READ TAP SWEEPS 48 → 48+depth SAMPLES BACK



tap = 48 + tri·depth >> 8

write W

Continuously varying delay detunes the wet copy — that shimmer mixed 50/50 with the dry signal is the chorus.

Per-sample (ISR)

```
tri = triangle(lfoPhase)
lfoPhase += rate
tap = 48 + (tri*depth>>8)
wet = buf[w - tap]

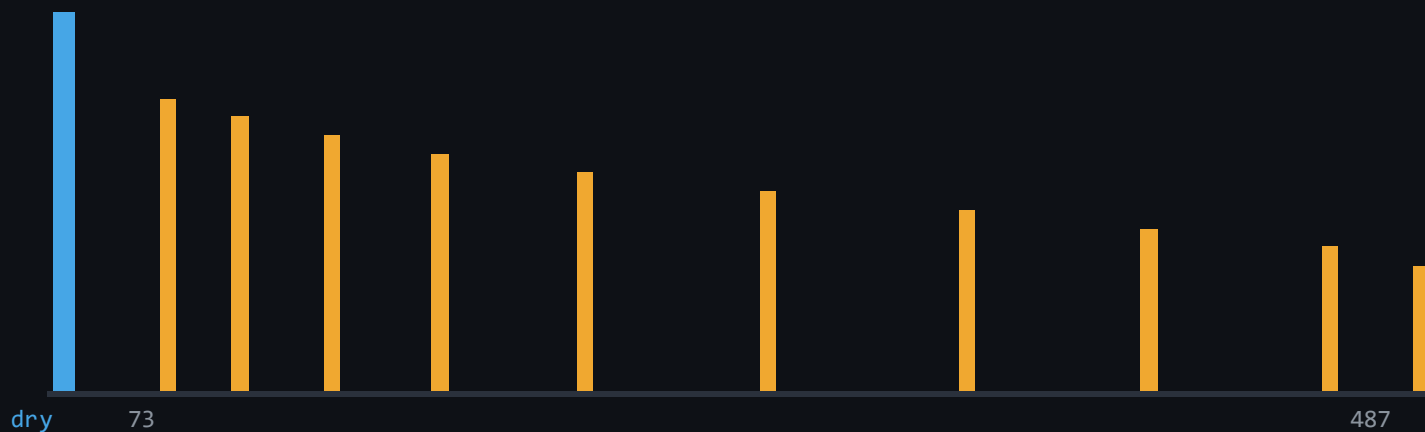
y = (x>>1) + (wet>>1)
// 50 / 50 dry+wet
// no feedback path
```

depth 1-48 samples

rate 1-20 · sets sweep speed

Reverb — 10 fixed taps

10 EQUAL-WEIGHT REFLECTIONS • `sum >> 3`



Taps 73 → 487 samples $\approx 7.6 - 50.7$ ms — dense pattern, not 10 discrete echoes

Per-sample mix

```
rev = (t1+t2+...+t10)
    >> 3;
store = x + rev*fb >> 8;
out   = x + rev*mix >> 8;
// recursive → dense tail
```

STABILITY GATE

$10 \cdot (1/8) \cdot fb/256 < 1$
→ feedback < 205, else self-oscillation.
Default 190 $\approx$ 300 ms half-life (medium room).

Tuner — AMDF pitch detection

$D(\tau)$ vs lag τ • deepest dip = fundamental period



THE MATH

$$D(\tau) = \sum |buf[n] - buf[n+\tau]|$$

- search $\tau = 24 \dots 128$
- parabolic interp → sub-sample
- $f = 9615 / \tau$ (deci-Hz)
- UI maps → E A D G B E

Runs in the main loop (~150 ms) — never in the ISR

Works on a frozen snapshot copied out of the ring buffer, so the ~9.6 kHz ISR can keep writing without smearing the AMDF mid-computation.

Auto-VGA — main-loop AGC

PEAK → GAIN RESPONSE

peak > 380

HIGH

step gain DOWN

40 - 380

DEF=15

hold (sweet spot)

8 - 40

LOW

step gain UP

peak < 8

NOISE

noise floor · never
boost

VGA loop · every 15 ms

```
if (peak > VGA_HIGH) {  
    if (OCR1A < MAX) OCR1A++; // ↓ gain  
} else if (peak > NOISE  
           && peak < LOW) {  
    if (OCR1A > 0) OCR1A--; // ↑ gain  
} else {  
    drift OCR1A → VGA_DEF (15)  
}
```

WHY ±1 STEP / 15 ms?

One OCR1A step per tick gives a smooth gain ride — no pumping or zipper. Any manual gain command switches auto-VGA off.

SECTION 03

Backend

FastAPI + Whisper STT + LLM tone engine

Backend architecture — FastAPI

API ENDPOINTS • `app/main.py`

GET `/api/state`
current AmpParams + effect name

POST `/api/params`
apply a full manual update

POST `/api/chat`
one tone-engine turn → board

POST `/api/transcribe`
browser audio → Whisper text

GET `/api/telemetry`
SSE stream • ~10 Hz

REQUEST → BOARD

AmpParams — canonical clamped state
models.py enforces every value range



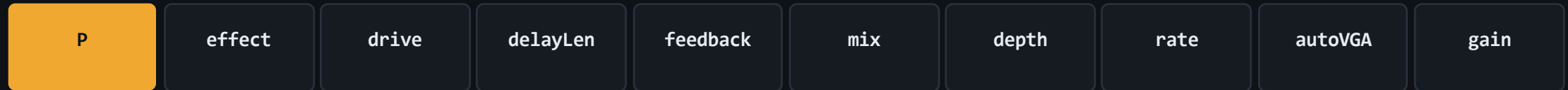
`apply_params()` — single funnel
lock → link.apply(p) → update state



SF4Link
real serial

Serial protocol — one atomic frame

HOST → MCU • set-all command (atomic — no ISR half-reads)



gain = -1 sentinel → leave the VGA (and auto-VGA mode) untouched

MCU → HOST • telemetry ~10 Hz



freqDeciHz present only in tuner mode (Hz × 10)

models.py • AmpParams.to_frame()

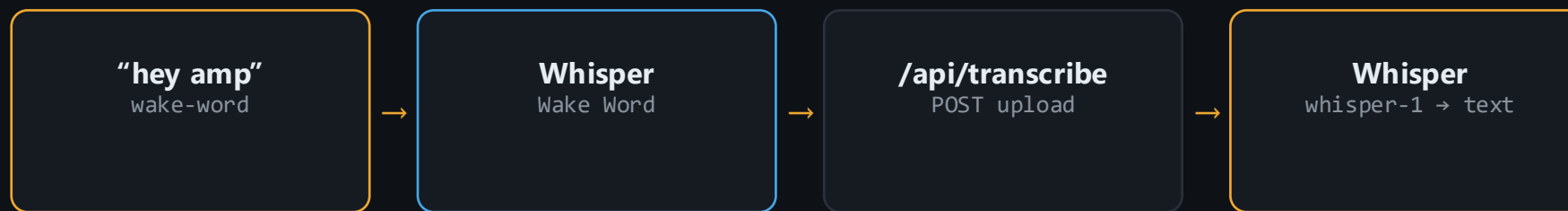
```
"P," + ",".join(str(v) for v in [
    int(effect), drive, delay_len, feedback,
    mix, depth, rate, 1 if auto_vga else 0,
    gain if gain is not None else -1 ])
```

CLAMPED RANGES

drive	1-511	depth	1-48
delayLen	1-512	rate	1-20
feedback	0-255	gain	0-255
mix	0-255	effect	0-5

Voice input — Whisper STT

VOICE PIPELINE



`/api/transcribe` (FastAPI)

```
raw = await file.read()
buf = io.BytesIO(raw)
buf.name = "recording.webm"
txt = client.audio.transcriptions.create(
    model="whisper-1", file=buf,
    language="en").text
```

NOTES

Returns text only

The transcript goes to the chat endpoint as a normal user turn.

Model configurable

OPENAI_TRANSCRIBE_MODEL env override.

LLM tone engine — system prompt

tone_engine.py · SYSTEM_PROMPT

You are the tone engine for the SF4 guitar amp.
You translate a player's request into amp settings
by calling the set_amp_params tool.

EFFECTS (one active at a time):

clean	- no processing
overdrive	- drive 1..511 (16 breakup, 180 rock, 480 metal)
delay	- delay_len 1..512, feedback, mix (t = delay_len/9615 Hz, <=53 ms)
chorus	- depth 1..48, rate 1..20
reverb	- feedback (KEEP < 205 or it self-oscillates), mix
tuner	- clean passthrough; UI shows EADGBE

auto_vga defaults true; manual gain 0..255.

GUIDELINES

- Tone-setting requests are ACTION requests, not questions -> you MUST call set_amp_params before replying.
- Pick the single best effect for the request.
- Change only the fields that matter; edits are relative to the CURRENT settings.
- Purely factual questions: answer, no tool.
- Keep spoken replies short.

KEY RULES

Action, not chat

"Tone-setting requests are action requests, not questions" — the model must call the tool before it speaks.

Relative edits

Patches merge onto current state — "more drive" nudges, not resets.

Guardrails in the prompt

reverb feedback < 205, one effect at a time — the LLM is told the safe ranges.

LLM tone engine — tools & flow

REQUEST → PARAMETERS



TOOL `set_amp_params`

```
"effect": clean | overdrive | delay |  
          chorus | reverb | tuner  
"drive":   int  1-511  
"delay_len": int 1-512  
"feedback": int 0-255  
"mix":      int 0-255  
"depth":    int 1-48  
"rate":     int 1-20  
"auto_vga": bool  
"gain":     int 0-255  
additionalProperties: false
```

→ every field optional; omitted = unchanged

Force-tool on tone keywords (`_TONE_ACTION_WORDS`) • `MAX_TOOL_ROUNDS = 4` • OpenAI gpt-5.5 (default) → Anthropic Claude fallback